Complexity Analysis

[CSE2010] Data Structures

Department of Data Science

Performance Evaluation

- Performance measurement (machine-dependent)

- Performance analysis (machine-independent)
  - Space complexity
  - Time complexity

Space Complexity

The amount of space (or memory) taken by the algorithm to run as a function of its input length

Space Complexity

- The amount of space (or memory) taken by the algorithm to run as a function of its input length
  - Include auxiliary space (and space used by the input).
  - Auxiliary space: the temporary or extra space the algorithm consumes
  - In this course, space complexity and auxiliary space will be interchangeably used.

- **if there are N data elements, how many units of memory will the algorithm consume?**

What is the auxiliary space of the code below?

```
def tripleInPlace(A: list[int]) -> list[int]:
    for i,_ in enumerate(A):
        A[i] = A[i] * 3
    return A
```

- Space complexity: I where I is the size of an integer

```
def tripleExtra(A: list[int]) -> list[int]:
    U = []
    for i,_ in enumerate(A):
        U.append(A[i] * 3)
    return U
```

- Space complexity: $I \times (n + 1)$ where I is the size of an integer and n is the number of elements in A

Time Complexity

The amount of time taken by an algorithm to complete its process as a function of its input length.

Time Complexity

- The amount of time taken by an algorithm to complete its process as a function of its input length.
- The number of primitive operations or steps executed in the algorithm
- **Time complexity, speed, efficiency, runtime, running time** interchangeably

- **if there are N data elements, how many steps will the algorithm take?**

What is the time complexity of the code below?

```
def getSum(A: list[int]) -> int:
    int total = 0
    for e in A:
        total += e
    return total
```

Number of Steps
1
$n+1$
n
1
Running time: $\boldsymbol{2n+3}$

Asymptotic Analysis

Given two algorithms with their running times,

which algorithm is better?

Asymptotic Analysis: Find Max

- In algorithm analysis, focus on the growth rate of the running time as a function of the input size
  - Running time of an algorithm grows proportionally to n.

```
def findMax(A):
    largest = A[0]
    for val in A:
        if val > largest:
            largest = val
    return largest
```

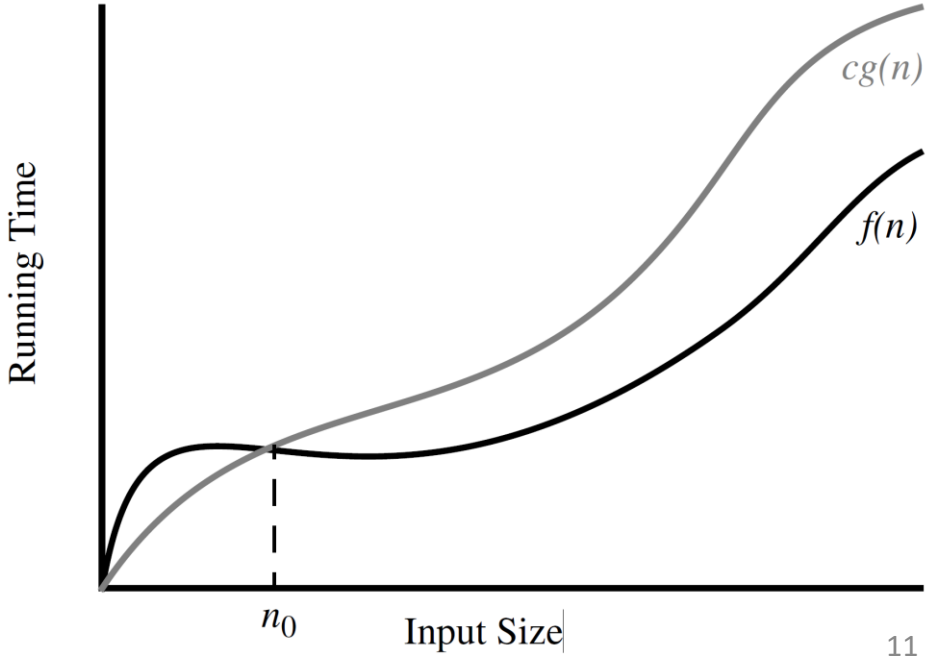
Big-Oh

- For a given function $g(n)$

$$O\big(g(n)\big) = \{f(n): there\ exist\ positive\ constants\ c\ and\ n_0\ such\ that\ 0 \le f(n) \le cg(n) for\ all\ n \ge n_0\}$$

- An algorithm takes $O\big(g(n)\big)$ if its running time is upper-bounded by $cg(n)$.
  - $O(n), O(nlogn), O(n^2), O(2^n), \ldots$

- Intuitively,
  - $f(n) \in O(g(n)) \Leftrightarrow$ f grows no faster than g



Example

- Show that $f(n) = 7n + 3$ is $O(n)$.

- Answer.
  There exist a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $7n + 3 \leq cn$ for every integer $n \geq n_0$.

$$c = 8 \text{ and } n_0 = 3$$

$$c = 10 \text{ and } n_0 = 1$$

$$\vdots$$

- Infinitely many choices available as there is a trade-off between c and n_0.

Example

- Show that $f(n) = 4n^3 + 9n^2 + 2n + 5$ is $O(n^3)$.

- Answer.

  There exist a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $4n^3 + 9n^2 + 2n + 5 \leq cn$ for every integer $n \geq n_0$.

  For all $n \geq n_0 = 1$, $4n^3 + 9n^2 + 2n + 5 \leq (4 + 9 + 2 + 5)n^3 = 20n^3$.

$$c = 20 \text{ and } n_0 = 1$$

Asymptotic Analysis: Find Max

- The number of instructions in findMax is $c_1 + c_2 n$.
- The algorithm findMax runs in O(n) time.

```
def findMax(A):
    largest = A[0] // constant c_1
    for val in A: // repeat n times
        if val > largest: // constant c_2
            largest = val
    return largest
```

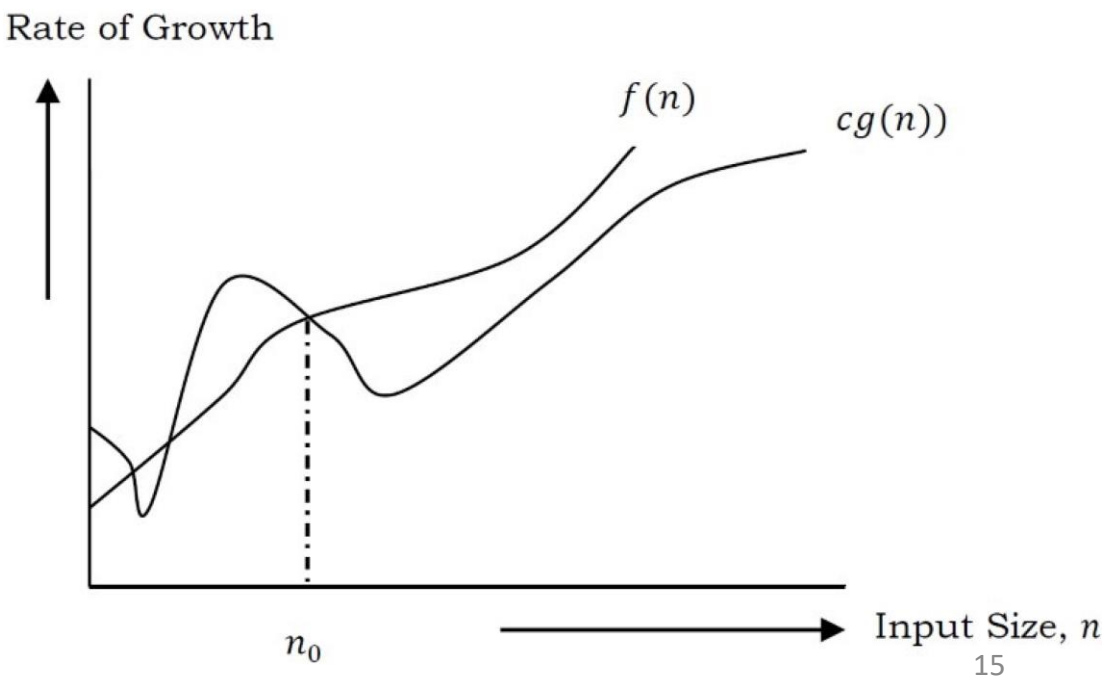
Big-Omega

- For a given function $g(n)$

$$\Omega\big(g(n)\big) = \{f(n): there\ exist\ positive\ constants\ c\ and\ n_0\ such\ that\ f(n) \geq cg(n)\ for\ all\ n \geq n_0\}$$

- An algorithm takes $\Omega\big(g(n)\big)$ if its running time is lower-bounded by $cg(n)$.

- Intuitively,
  - $f(n) \in \Omega(g(n)) \Leftrightarrow$ f grows no slower than g



Example

- Show that $4nlogn - 3n$ is $\Omega(nlogn)$.

- Answer.

  There exist a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $4nlogn - 3n \leq cnlogn$ for every integer $n \geq n_0$.

  $4nlogn - 3n = nlogn + 3n(logn - 1) \geq nlogn$ for $n \geq 2$.

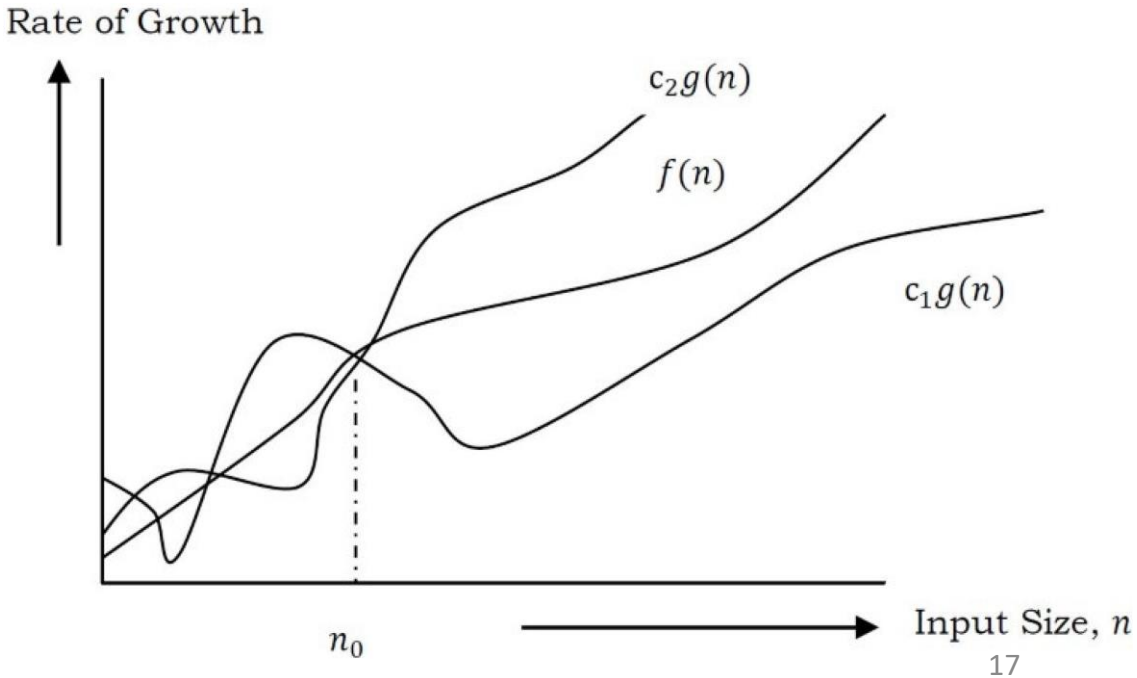
$$c = 1 \text{ and } n_0 = 2$$

Big-Theta

- For a given function $g(n)$
  $\Theta\big(g(n)\big) = \{f(n)\colon there\ exist\ positive\ constants\ c_1, c_2, and\ n_0$
  $such\ that\ \ c_1 g(n) \le f(n) \le c_2 g(n)\ for\ all\ n \ge n_0\}$

- An algorithm takes $\Theta\big(g(n)\big)$ if
  its running time is lower-bounded by $c_1 g(n)$
  and upper-bounded by $c_2 g(n)$.

- More precise than the big-oh and big-omega notations.



Comparative Analysis

- Suppose two algorithms A and B are given:
- A has a running time of $O(n)$.
- B has a running time of $O(n^2)$.
- A is asymptotically better than B

- Functions ordered by increasing growth rate:
  - If a function f(n) precedes a function g(n), then f(n) is O(g(n))

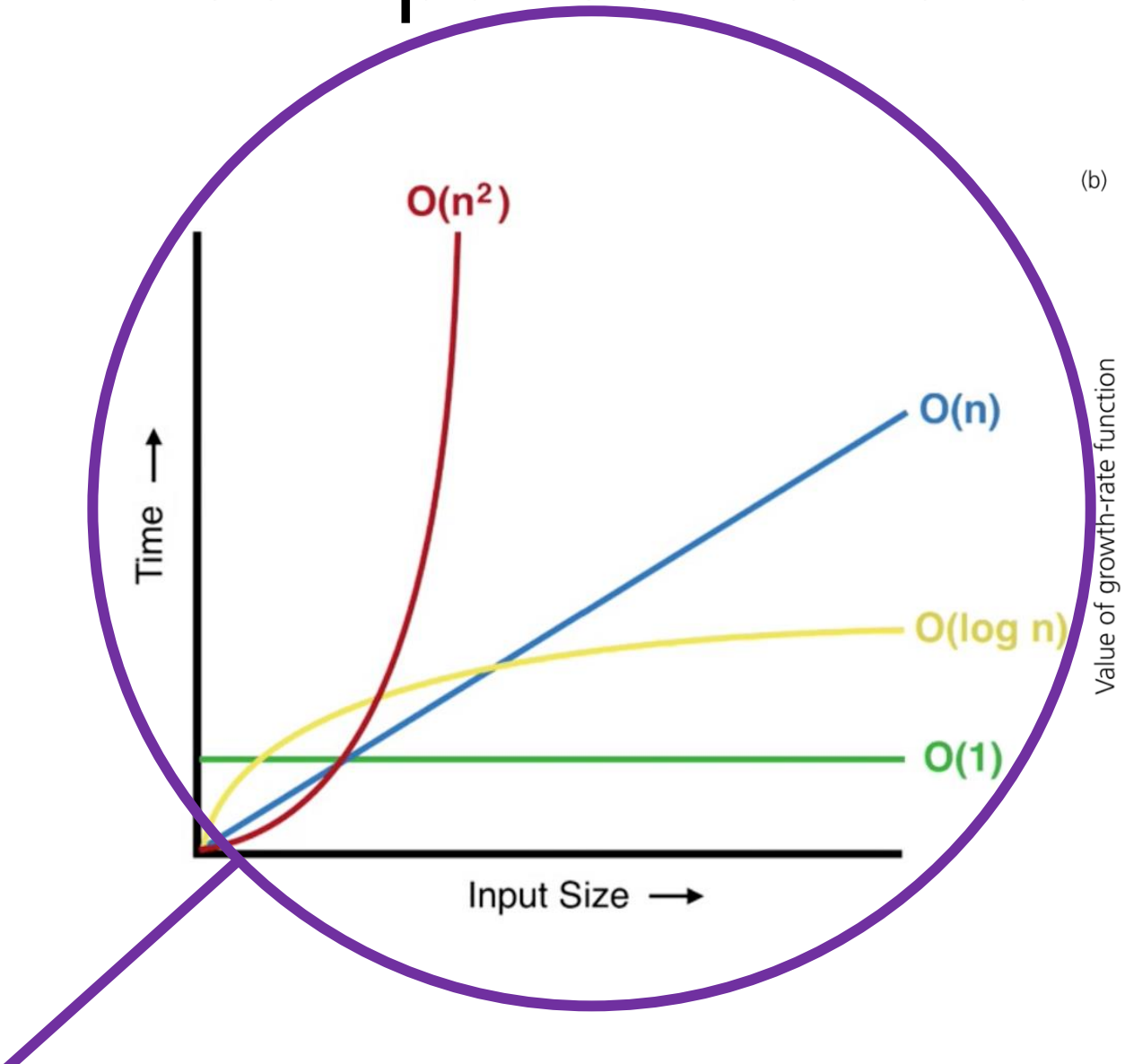
$$1, \quad logn, \quad n, \quad nlogn, \quad n^2, \quad n^3, \quad 2^n$$

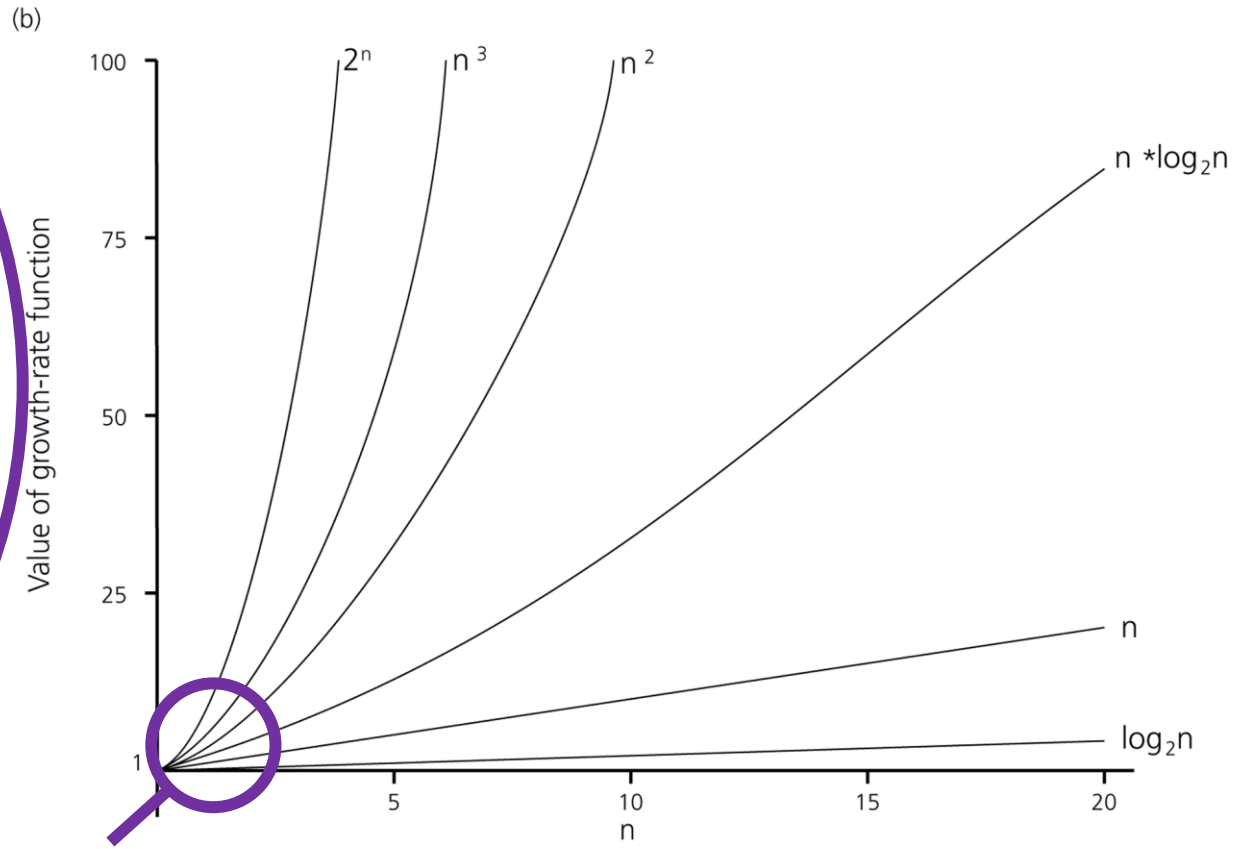
Comparison of Growth-Rate Functions

- Selected values of fundamental functions in algorithm analysis

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,096	262,144	1.84×10^{19}
128	7	128	896	16,384	2,097,152	3.40×10^{38}
256	8	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	512	4,608	262,144	134,217,728	1.34×10^{154}

Comparison of Growth-Rate Functions





Comparative Analysis

- Assume 10^9 steps/sec in a computer
- Program A takes n^3 steps for execution
- Program B takes 2^n steps for execution

n	n^3	2^n
32	32 µs	4.3 s
128	2 ms	6.5×10^{23} years
512	0.1 s	2.5×10^{139} years

Types of Running-Time Analysis

- Best-case analysis
  - Analysis for the best-case input
  - Mostly not meaningful
- Worst-case analysis
  - Analysis for the worst-case input
- Average-case analysis
  - Analysis for all inputs
  - More difficult to analyze

Guidelines for Asymptotic Analysis

- Loops:

$$c \times n \in O(n)$$

- Nested Loops:

$$c \times n \times n \in O(n^2)$$

```
int m = 0;
for(int i = 1; i < n; ++i)
    m = m + 1; // constant time, c
```

```
int m = 0;
for(int i = 1; i < n; ++i){
    for(int j = 1; j <= n; ++j)
        m = m + 1; //constant time, c
}
```

Guidelines for Asymptotic Analysis

- Consecutive Statements:

$$c_0 + c_1 n + c_2 n^2 \in O(n^2)$$

```
int x = 0, y = 0;
x = x + 3; // c_0
for(int i = 0; i < n; ++i)
    x = x + 2; // c_1
for(int i = 0; i < n; ++i)
    for(int j = 0; j < n; ++j)
        y = y + 1; // c_2
```

- If-Then-Else Statements:

$$c_0 + (c_2 + c_3)n \in O(n)$$

```
if(length() == 0){// c_0
    return false;  // c_1
}
else{
    for(int i = 0; i < length(); ++i){
        if(A[i] == B[i]) // c_2
            return false; // c_3
    }
}
```

Example: Prefix Averages

- Given an array S consisting of n numbers, compute an array A such that $A[j]$ is the average of elements $S[0], \dots, S[j]$, for $j = 0, \dots, n-1$:

$$A[j] = \frac{\Sigma_{i=0}^{j} S[i]}{j+1}$$

Example: Prefix Averages

```
void prefixAverage(int* S, int* A, int n){
    for(int j = 0; j < n; ++j){// c_1
        int total = 0; // c_2
        for(int i = 0; i < j+1; ++i)
            total += S[i]; // c_4
        A[i] = total / (j+1); // c_5
}
```

Number of Steps
$c_1(n+1)$
$c_2 n$
$c_3(2+3+\cdots+n+(n+1))$
$c_4(1+2+3+\cdots+n) = c_4 n(n+1)/2$
$c_5 n$
Running time: $\boldsymbol{O(n^2)}$

Example: Prefix Averages

```
void prefixAverageLinear(int* S, int* A, int n){
    int total = 0; // c_1
    for(int j = 0; j < n; ++j){ //c_2
        total += S[j]; // c_3
        A[i] = total / (j+1); // c_4
}
```

Number of Steps
c_1
$c_2(n+1)$
$c_3 n$
$c_4 n$
Running time: $O(n)$